

Modul 12 – Refactoring & Navigasi Multi Halaman

A. Tujuan Pembelajaran

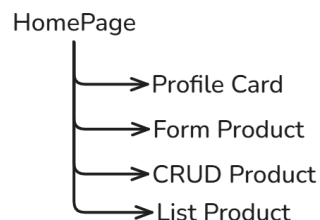
Setelah mengikuti praktikum ini, mahasiswa mampu:

1. Memahami pentingnya Refactoring dalam pengembangan aplikasi.
2. Memisahkan kode berdasarkan tanggung jawab (*Separation of Concerns*).
3. Melakukan navigasi antar halaman menggunakan Navigator.
4. Mengirim data antar halaman (*Passing Data*).
5. Membuat widget yang dapat digunakan kembali (*Reusable Widget*).

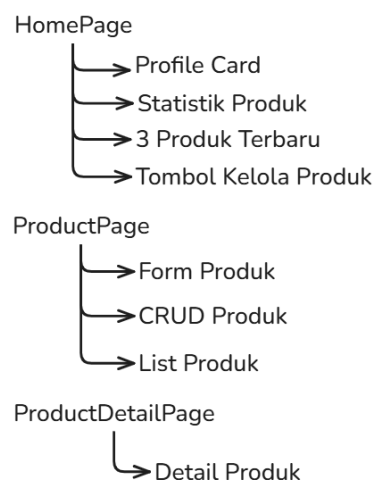
B. Dasar Teori

Refactoring merupakan proses memperbaiki struktur kode tanpa mengubah fungsi aplikasi.

Jika pada sebelumnya struktur kode yang berada di HomePage seperti berikut:

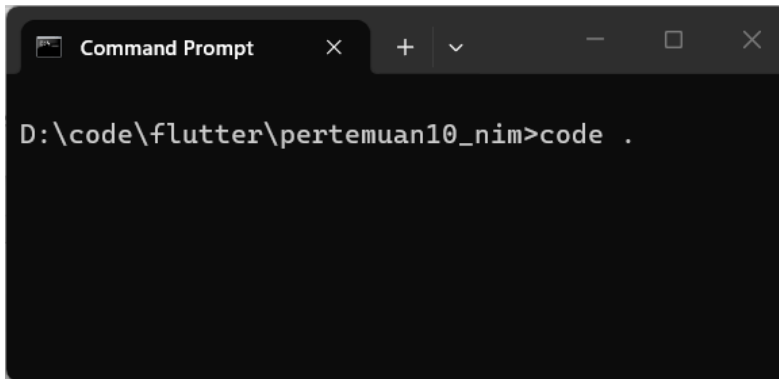


Maka akan dirubah menjadi seperti berikut.



C. Praktikum

1. Silahkan buka kembali project flutter **pertemuan10_nim** ke dalam VS Code.



2. Jalankan project flutter tersebut lewat terminal dengan perintah **flutter run -d chrome --web-port=3000** dengan begitu port dari emulator chrome akan konsisten berjalan di port 3000.
3. Buka file lib/pages/home_page.dart.
4. Tambahkan kode berikut untuk mengambil 3 produk terbaru.

```
Future<void> loadProducts() async {  
  final prefs = await SharedPreferences.getInstance();  
  List<String> productList = prefs.getStringList('products') ?? [];  
  totalProducts = productList.length;  
  setState(() {  
    products = productList  
      .reversed  
      .take(3)  
      .map((item) => ProductModel.fromJson(item))  
      .toList();  
  });  
}
```

Sebelum mapping data tambahkan `.take(3)` untuk memberikan Batasan agar data yang diambil yaitu 3.

5. Inisialisasi variable berikut untuk menampilkan total produk

```
int totalProducts = 0;
```

6. Tambahkan widget untuk menampilkan total data dan juga widget untuk tombol pindah halaman ke daftar produk

```
Text(
  "Total Produk : ${totalProducts.toString()}",
  style: const TextStyle(fontWeight: FontWeight.bold),
), // Text
// tombol pindah halaman
TextButton(
  onPressed: () {
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => const ProductPage(),
      ), // MaterialPageRoute
    );
  },
  child: const Text("Lihat selengkapnya"),
), // TextButton
```

Simpan kedua widget tersebut ke dalam widget Row.

7. Save dan hot restart aplikasi yang dijalankan pada emulator.
8. Buat file baru pages/product_page.dart.
9. Buat class state _ProductPageState didalamnya

```
class ProductPage extends StatefulWidget {
  const ProductPage({super.key});

  @override
  State<ProductPage> createState() => _ProductPageState();
}

class _ProductPageState extends State<ProductPage> {
  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

10. Berikutnya buat appBar terlebih dahulu untuk halaman daftar produk ini

```
appBar: AppBar(
  title: const Text("Produk", style: TextStyle(color: Colors.white)),
  backgroundColor: Colors.green,
  leading: IconButton(
    icon: Icon(
      Icons.arrow_back,
      color: Colors.white,
    ), // Icon
    onPressed: () => Navigator.pop(context),
  ), // IconButton
), // AppBar
```

11. Inisialisasi variable product di class state _ProductPageState

```
List<ProductModel> products = [];
```

12. Salin method loadProduct() dari home_page ke product_page.

```
Future<void> loadProducts() async {  
  final prefs = await SharedPreferences.getInstance();  
  List<String> productList = prefs.getStringList('products') ?? [];  
  setState(() {  
    products = productList  
      .map((item) => ProductModel.fromJson(item))  
      .toList();  
  });  
}  
  
@override  
void initState() {  
  super.initState();  
  loadProducts();  
}
```

Kemudian pindahkan juga method saveProduct(), addProduct(), editProduct() dan deleteProduct() ke product_page.

13. Pada bagian property body gunakan Container sebagai widget utamanya.

```
body: Container(  
  margin: const EdgeInsets.all(20),  
  child: Column(  
    children: [  
      .  
    ],  
  ), // Column  
) , // Container
```

Pindahkan widget tombol tambah produk dan daftar produk dari home_page ke dalam product_page

14. Sekarang silahkan pastikan tidak ada code error di home_page dan juga di product_page. Jika sudah save project flutter kemudian jalankan lewat terminal lagi.
15. Selanjutnya buat file baru pages/product_detail_page.dart.

16. Buat class ProductDetailPage yang extends ke StatelessWidget dan buat parameter variable berikut dengan contructornya

```
final ProductModel product;  
  
const ProductDetailPage({super.key, required this.product});
```

Untuk widget builder nya cukup buat seperti code dibawah.

```
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: const Text("Detail Produk")),  
    body: Padding(  
      padding: const EdgeInsets.all(20),  
      child: Column(  
        crossAxisAlignment: CrossAxisAlignment.start,  
        children: [  
          Text(  
            product.name,  
            style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold),  
          ), // Text  
          const SizedBox(height: 10),  
          Text("Rp ${product.price}"),  
          const SizedBox(height: 10),  
          Text(product.description),  
        ],  
      ), // Column  
    ), // Padding  
  ); // Scaffold  
}
```

17. Kemudian membuat reusable widget, jadi buat folder baru yaitu widgets dan tambahkan file baru product_card.dart.

```
class ProductCard extends StatelessWidget {  
  
  @override  
  Widget build(BuildContext context) {  
    return Container();  
  }  
}
```

Tambahkan variable sebagai parameter dari product card

```
// membuat variable dari parameter product  
final ProductModel product;  
final VoidCallback onTap;  
final VoidCallback? onEdit;  
final VoidCallback? onDelete;
```

Dan juga constructor nya

```
// constructor untuk memanggil parameter
const ProductCard({
  super.key,
  required this.product,
  required this.onTap,
  this.onEdit,
  this.onDelete,
});
```

Setelah itu didalam widget builder pindahkan code return Card yang ada di product_page ke dalam product_card, hanya saja yang membedakan yaitu tambah property onTap didalamnya dan juga ubah property leading dan trailing seperti berikut.

```
leading: onEdit != null ? IconButton(
  icon: const Icon(Icons.edit, color: Colors.orange),
  onPressed: onEdit,
): null, // IconButton
trailing: onDelete != null ? IconButton(
  icon: const Icon(Icons.delete, color: Colors.red),
  onPressed: onDelete,
) : null, // IconButton
```

18. Buka kembali file pages/product_page.dart dan ubah bagian return Card menjadi berikut.

```
return ProductCard(
  product: product,
  onDelete: () => deleteProduct(index),
  onEdit: () =>
    showForm(product: product, index: index),
  onTap: () => Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) =>
        ProductDetailPage(product: product),
    ), // MaterialPageRoute
  ),
); // ProductCard
```

Sehingga sekarang card product sudah reusable (digunakan berulang) dimana saja. Dan hanya cukup mengirimkan parameter yang dibutuhkannya saja. Untuk home_page.dart juga silahkan return Card nya ubah menggunakan ProductCard.